《深度学习》

Chap6 循环神经网络实验报告

基于 RNN 的唐诗生成系统及跨框架性能对比分析

学号：2351407

姓名：楼思含

专业：数据科学与大数据技术

2026 年 3 月 22 日

目 录

一、实验目的.....1
二、实验内容.....1
三、相关模型原理阐述.....1
3.1 RNN（循环神经网络）.....1
3.2 LSTM（长短期记忆网络）.....2
3.3 GRU（门控循环单元）.....2
四、实验步骤.....3
4.1 数据预处理.....3
4.2 模型构建与训练.....3
4.3 诗歌生成.....3
五、实验结果与性能表现.....4
5.1 框架架构对比与性能耗时分析.....4
5.2 诗歌生成过程.....4
5.2.1 pytorch.....5
5.2.2 tensorflow.....6
5.3 实验结果.....7
5.3.1 pytorch.....8
5.3.2 tensorflow.....8
六、实验总结.....9
6.1 实验总结与心得体会.....9
6.2 核心问题与解决方案.....9
6.2.1 无法追踪权重.....9
6.2.2 截断输出.....10
七、参考文献.....11

一、实验目的

本实验旨在通过构建并训练深度学习模型，深入理解自然语言处理中序列到序列或自回归生成任务的核心机制。在此过程中，要求熟练掌握标准循环神经网络（RNN）、长短期记忆网络（LSTM）以及门控循环单元（GRU）的数学原理与结构差异。本实验分别基于 PyTorch 与 TensorFlow 两大主流深度学习框架完成了核心代码的编写与模型训练，旨在通过平行实验对比二者在底层架构设计、动态与静态图机制以及实际运算效率（如训练耗时）上的显著差异。

二、实验内容

本次实验的核心任务紧密围绕理论与实践展开。在理论层面，需要详尽系统性地了解 RNN、LSTM 以及 GRU 这三种主流时序模型的内部运作机制与演进脉络，并梳理基于深度神经网络进行诗歌生成的全流程原理。在实践层面，实验分别在 PyTorch 环境下补全了 rnn.py 中的双层 LSTM 结构与前向传播逻辑，在 TensorFlow 环境下完善了 myRNNModel 的构建与使用 GradientTape 的自定义训练循环代码。在完成两个版本模型的漫长训练后，以“日、红、山、夜、湖、海、月”这七个特定的汉字作为起始序列引导模型生成相应的唐诗。最终整合上述理论分析。

三、相关模型原理阐述

在处理如诗歌这类具有强烈前后文依赖关系的序列数据时，传统的全连接神经网络由于缺乏时序记忆能力而显得捉襟见肘，由此催生了循环神经网络及其一系列变体模型。

3.1 RNN（循环神经网络）

RNN 的核心创新在于其网络层内部引入了自回环结构，使得隐层状态不仅取决于当前时刻的输入，还直接受到上一时刻隐层输出的影响，这种机制赋予了网络处理变长序列数据的记忆能力。

在时间步 t，RNN 的隐藏状态更新公式为$h_t = tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t + b)$，其中 W_{hh} 和 W_{xh} 是网络需要学习的权重矩阵，它将历史信息和当前输入拼接，通过 $tanh$ 激活函数将其压缩到 -1 到 1 之间。

然而，在标准的 RNN 结构中，由于梯度在反向传播时需要随时间步连续相乘，这导致其在处理长序列（如长篇诗歌或文章）时极易遭遇梯度消失或梯度爆炸的问题，从而难以捕获长距离的语义依赖。如果权重 <1，连乘多次后梯度会无限趋近于 0（梯度消失），模型会彻底忘记很久之前的信息；如果权重 >1，梯度会呈指数级爆炸（梯度爆炸），导致模型崩溃。

3.2 LSTM（长短期记忆网络）

为了克服标准 RNN 在长程记忆上的固有缺陷，LSTM 引入了复杂的“细胞状态”作为贯穿整个序列的信息传送带，并设计了三个门控机制：遗忘门、输入门和输出门。

隐藏状态 h_t 是短期记忆，也是当前时间步的输出。细胞状态 C_t 为长期记忆，直接贯穿整个序列，只有极少的线性交互，保证了梯度可以顺畅地流传很久而不消失。

门控本质上是一个 Sigmoid 函数，输出为 0 到 1 之间的值，用来控制信息通过的比例，0 代表完全阻断，1 代表完全通过。遗忘门负责决定哪些历史信息需要被丢弃，它查看前一个状态 h_{t-1} 和当前输入 x_t，输出一个 0~1 的向量，去乘以过去的细胞状态 C_{t-1}，如果遇到句号，它可能会决定“忘记”上一句的主语；输入门负责筛选当前时刻的新信息并将其更新至细胞状态中，它同样基于 h_{t-1} 和 x_t 计算出哪些新内容是有价值的，并结合新生成的候选记忆 $\widetilde{C}_t$，将其加到细胞状态中；输出门则基于当前的细胞状态计算出最终的隐层输出，它基于更新后的细胞状态 C_t，决定当前时间步应该把哪些记忆作为隐藏状态 h_t 暴露出去，用于预测下一个字。这种精密的控制机制有效缓解了梯度消失现象，使其成为自然语言生成领域的基石模型。

3.3 GRU（门控循环单元）

GRU 可以被视作 LSTM 的一种轻量化变体架构。它将 LSTM 中的遗忘门与输入门合并为一个单一的“更新门”，并保留了用于控制历史信息流入程度的“重置门”。

更新门 z_t 将 h_t 和 C_t 直接合并成了一个单一的隐藏状态 h_t，直接决定了“保留多少旧记忆”与“引入多少新记忆”的比例。重置门 r_t 它决定了在计算当前的新候选记忆时，要忽略多少过去的隐藏状态，如果重置门接近 0，网络就会丢弃历史信息。

由于去除了独立的细胞状态并减少了门控单元的数量，且张量维度更小，GRU 在保持与 LSTM 相当的序列建模性能的同时，大幅度降低了模型的参数量与计算复杂度，在与 LSTM 性能几乎一致的情况下训练时间大幅缩短，在训练资源有限或追求高推理速度的场景下展现出了极大的优势。

四、实验步骤

本实验的唐诗生成过程是一个自回归的语言建模任务，跨越了从原始数据清洗到模型推理输出的多个步骤。

4.1 数据预处理

首先，在数据预处理阶段，系统读取了海量的唐诗文本文档，剔除了其中包含特殊符号或长度不合规的异常数据，并在每首合格诗歌的句首与句尾分别拼接入人为定义的起始符与终止符。随后，算法对所有出现的汉字进行词频统计以构建全局字符映射表，将文本格式的诗歌转化为模型可读的离散整型索引序列。

4.2 模型构建与训练

其次，在模型构建与训练阶段，输入的离散索引序列首先经过词嵌入层转化为高维且连续的稠密向量，以捕捉汉字之间的潜在语义关联。在 PyTorch 版本中，这些向量被送入 nn.LSTM 模块（双层 LSTM 核心网络）中提取复杂的时序特征，并配合 optim.RMSprop 与 NLLLoss 在 Python 循环中进行动态梯度更新，网络的输出最终连接至一个全连接层与 LogSoftmax 激活函数，用于计算词表中每一个汉字作为下一个字符出现的概率分布，在训练循环中，算法通过计算预测分布与真实后继字符之间的负对数似然损失，并利用优化器计算梯度来反向更新网络权重，使模型逐步学习到唐诗的押韵规则与语法结构；而在 TensorFlow 版本中，则利用了 tf.keras.layers.SimpleRNNCell 构建循环核心，并通过 @tf.function 装饰器将基于 GradientTape 的动态求导过程编译为高效的静态计算图以加速优化。

4.3 诗歌生成

最后，在诗歌生成阶段，为了使模型能够根据指定的起始词汇进行创作，系统首先将起始符与该指定词汇拼接构成初始上下文，并将其转化为张量输入至加载了预训练权重的网络中。模型输出序列末尾概率最高的字符被提取并转换回汉字，随后该新生成的汉字被追加至当前序列中再次作为输入馈入模型。这一循环预测过程持续进行，直到模型预测出代表结束的终止符或达到预设的最大序列长度，从而完成一首完整唐诗的自动生成。

五、实验结果与性能表现

5.1 框架架构对比与性能耗时分析

由于 PyTorch 与 TensorFlow 两大框架有着截然不同的设计调度机制，因此二者虽然在最终生成文本的韵律与语义连贯性上表现相当，但在系统运算效能上却呈现出显著差异：PyTorch 框架下的模型训练耗时数小时之久，而 TensorFlow 框架下的模型训练仅耗时四十六分钟即宣告完成。

首先，在数据加载层面，本实验的 TensorFlow 版本深度调用了 tf.data.Dataset API，该组件在底层 C++ 层面实现了高度优化的异步数据预取、随机打乱以及高效的动态张量填充，极大程度地消除了 GPU 显存空载等待 CPU 喂数据的时间瓶颈；反观本实验的 PyTorch 版本，其数据批处理函数 generate_batch 主要依赖于原生的 Python 列表切片与循环拼接逻辑，这种纯 Python 层面的数据迭代在面临海量序列数据时，产生了高昂的解释器开销与数据搬运延迟。

其次，在图编译与算子优化层面，TensorFlow 版本巧妙地在自定义训练步函数 train_one_step 上方使用了 @tf.function 装饰器，这一操作触发了 AutoGraph 机制，将原本属于动态执行的 Python 训练逻辑瞬间编译为了静态的、经过 XLA 优化的底层计算图，使得反向传播与梯度更新脱离了 Python 解释器的束缚；而 PyTorch 则秉持了极致的“动态计算图”原则，每一次前向与反向传播都在运行时即时构建与销毁计算图，虽然赋予了极高的调试自由度(如方便插入断点)，但在未进行严格的算子融合与并行优化的前提下，其包含在多层 for 循环中的张量切片与拼接操作不可避免地拉长了整体的训练周期。

5.2 诗歌生成过程

5.2.1 pytorch

该诗歌生成系统采用了自回归序列生成策略，其核心原理是利用带有两层长短期记忆网络（LSTM）的深度学习架构来捕捉语言的时序依赖关系。整个诗歌的自动生成过程可以划分为数据与模型初始化、初始上下文构建、循环预测解码以及输出后处理四个连贯的阶段。

首先，在数据与模型初始化阶段，系统调用 process_poems1 函数对原始唐诗文本文档进行预处理，通过剔除包含特殊符号或长度不合规的废弃语料，并在每首合格诗歌的首尾分别拼接入人为定义的起始符（G）与终止符（E），进而统计字频并构建出用于字符与整型索引相互转换的字典与查找表。依托于该词汇表的维度信息，系统实例化了自定义的 RNN_model 神经网络，该模型在架构上依次包含了用于将离散索引降维映射为稠密语义向量的词嵌入层、用于解决长程梯度消失问题并提取深层时序特征的双层 LSTM 网络、用于将隐层状态投射回词汇表维度的全连接层，以及最终输出归一化概率分布的 LogSoftmax 层。在结构定义完成后，系统通过 load_state_dict 接口将前期耗费大量算力训练并落地的预训练权重文件加载至内存中，使得模型瞬间具备了对古典唐诗语法与平仄规律的先验知识。

```
self.rnn_lstm = nn.LSTM(input_size=embedding_dim,
                        hidden_size=lstm_hidden_dim,
                        num_layers=2,
                        batch_first=True)
```

其次，在进入具体的生成任务即上下文构建阶段时，为了使模型能够精准地按照提供的特定汉字（如“日”、“红”、“山”等）开启创作，算法并未生硬地将孤立的单字直接喂入网络，而是严格遵循了模型在训练期所见的数据分布形态，将系统定义的全局起始符‘G’与指定的起始汉字进行字符串拼接，从而构建出诸如“G 日”这样包含结构引导信息的初始上下文序列，以此作为唤醒模型连续生成机制的“种子”语料。

随后，系统便切入最为核心的循环预测解码环节。在代码的 while 循环体中，当前的文本序列首先经由映射字典被逐字转化为整型数组，并封装成 PyTorch 专属的 Variable 张量类型，随后在指定 is_test=True 的推理模式下被送入模型进行前向传播计算。在模型内部，输入张量依次流经词嵌入空间与双层 LSTM 记忆单元，网络仅截取最后一个时间步的隐藏状态特征，使其穿过全连接层与 ReLU 激活函数，最终计算出涵盖所有词汇的对数概率预测分布。紧接着，辅助函数 to_word 运用贪心搜索策略（基于 np.argmax 操作）从该分布中筛选出发生概率最高的汉字索引并逆向解析为文本字符，随后将此新字符动态追加至先前的诗歌序列末尾；这一包含最新预测结果的延长序列会再次作为全新的输入张量馈入网

络，通过这种将上一步输出作为下一步输入的循环机制，模型得以源源不断地实现词汇的递进式生成。

```
output, (h_n, c_n) = self.rnn_lstm(batch_input)
```

最后，在终止控制与输出后处理阶段，上述的逐字迭代生成过程并非无休止地运行，而是受到双重停止条件的严格约束，一旦模型在某个时间步预测出了代表语句完结的终止符‘E’，抑或是生成的累计字符串长度触及了预设的三十个字符的安全阈值，解码循环便会被立刻中断。在最终向控制台交付生成结果之前，系统通过字符串切片操作（poem[1:]）静默移除了仅作引导之用且不具实际语义的起始占位符‘G’，并将这串连续的文本移交给 pretty_print_poem 函数，由其依据句号标识符将长字符串物理切割为多行子句，并滤除掉长度不达标的无效片段，最终输出结构规整、逻辑连贯且符合指定起始词汇的定制化唐诗作品。

5.2.2 tensorflow

该诗歌生成系统展现了基于 TensorFlow 2.x 动态图与静态图混合机制的自回归文本建模策略，其核心生成逻辑可清晰地划分为数据预处理与流水线构建、模型架构与计算图编译、循环优化训练以及引导式推理生成四个循序渐进的阶段。

首先，在数据预处理与流水线构建阶段，系统通过自定义的 process_dataset 函数逐行读取文本文档中的原始语料，并在每首满足长度约束（不超过 200 字）的诗歌序列首尾分别无缝嵌入起始符 bos 与终止符 eos。随后，算法基于全局字符的词频统计构建了从高频到低频排列的词汇表，并特意前置填充了用于序列对齐的 PAD 标记与处理未知字符的 UNK 标记，进而生成字符与离散整型索引的双向映射字典（word2id 与 id2word）。为了实现高效的数据馈入，系统利用 tf.data.Dataset.from_generator 接口构建了异步数据流水线，通过连续调用 shuffle 进行样本打乱、利用 padded_batch 将不同长度的诗歌补齐至同一批次的最大长度，并最终通过 map 函数将序列错位切分，构造出用于自回归预测的输入序列特征（x[:, :-1]）与错后一个时间步的监督标签序列（x[:, 1:]）。

其次，在模型架构与计算图编译阶段，系统通过继承 keras.Model 基类构建了定制化的 myRNNModel 神经网络结构。该模型在物理前向传播路径（call 方法）上依次串联了三个核心组件：首先是用于将高维稀疏的独热编码降维映射为包含 64 个特征维度的连续语义向量的 Embedding 层；其次是作为序列记忆核心、包含 128 个隐藏单元的 SimpleRNNCell

（被包裹于高阶接口 RNN 中以实现全序列输出）；最后是负责将隐层特征空间线性投射回词表维度以计算全局概率对数的 Dense 全连接层。为了兼容最新的框架特性并规避延迟初始化带来的权重追踪失败问题，系统在正式进入训练循环前，巧妙地注入了一个形态为 [1, 2, 3, 4] 的微型常数哑变量张量让网络进行一次空载前向传播，从而强制框架在内存中提前实例化所有的权重参数矩阵。

```
    def call(self, inp_ids):
        inp_emb = self.embed_layer(inp_ids)        # 词嵌入特征转换
        h = self.rnn_layer(inp_emb)                # RNN 层提取时序特征
        logits = self.dense(h)                     # 全连接层映射到词表大小
        return logits
```

紧接着，在循环优化训练阶段，借助 TensorFlow 强大的 tf.GradientTape 上下文管理器来精确捕捉并记录前向传播过程中所有可训练变量的计算轨迹。在单步训练函数 train_one_step 中，模型基于交叉熵算法比对预测概率与真实标签，并调用专门设计的辅助函数 reduce_avg 结合 sequence_mask 对序列中用于补齐的无效 PAD 部分进行掩码过滤，从而计算出纯净的序列平均损失。该单步训练函数及损失计算函数均被覆以 @tf.function 装饰器，这一操作触发了底层的 AutoGraph 引擎，将原本依赖 Python 解释器动态执行的代码瞬间编译为高度优化的静态计算图，显著提升了梯度求导与 Adam 优化器参数更新的执行效率。

最后，在引导式的推理生成阶段，代码设计了专门用于单步解析的 get_next_token 方法以及负责时序统筹的 gen_sentence 函数。不同于训练时的一股脑输入，在生成伊始，系统首先利用均值为 0、标准差为 0.5 的正态分布噪声随机初始化循环单元的隐藏状态，以确保生成的诗歌具有足够的多样性。随后，系统先向模型馈入起始标记 bos 以激活内部状态的更新，紧接着强行将用户指定的主题汉字（如“日”、“山”等）转化为索引作为下一个绝对输入，从而确立了整首诗歌的“藏头”基调。在后续的自循环解码过程中，上一步预测出的字符索引会不断经由 get_next_token 方法结合当前隐层状态生成下一步的字符，直至触发长达 50 个循环的强制长度上限，或是模型顺应语境提前输出表征完结的 eos 终止符才主动跳出循环。最终，收集到的索引数组被逆向映射回汉字字符并拼接为直观的字符串，由此完成了一首蕴含特定字词的诗词创作。

5.3 实验结果

5.3.1 pytorch

```
(.venv) PS C:\2351407-vscode\deep_learning\homework\chap_6\tangshi_for_pytorch> python main.py
error
finish  loadding data
inital  linear weight
C:\2351407-vscode\deep_learning\.venv\lib\site-packages\torch\nn\modules\module.py:1773: UserWarning: Implicit dimension choice for log_softmax has been deprecated. Change the ca
ll to include dim=X as an argument.
  return self._call_impl(*args, **kwargs)
prediction [2057, 3132, 2057, 4326, 4326, 2057, 3132]
b_y        [28, 546, 104, 718, 1, 3, 3]
******************************
epoch   0 batch number 0 loss is:  8.7185640335083
C:\2351407-vscode\deep_learning\homework\chap_6\tangshi_for_pytorch\main.py:164: FutureWarning: `torch.nn.utils.clip_grad_norm` is now deprecated in favor of `torch.nn.utils.clip
_grad_norm_`.
  torch.nn.utils.clip_grad_norm(rnn_model.parameters(), 1)
finish  save model
prediction [1, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
b_y        [267, 2888, 267, 2045, 0, 26, 1067, 26, 349, 1, 3, 3]
******************************
epoch   0 batch number 1 loss is:  7.504136562347412
prediction [588, 15, 208, 15, 99, 2329, 15, 2064, 2064, 2329, 2064, 77, 2329, 2064]
b_y        [617, 564, 80, 601, 132, 0, 9, 184, 2007, 373, 305, 1, 3, 3]
******************************
epoch   0 batch number 2 loss is:  8.290597915649414
prediction [22, 0, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10]
b_y        [125, 9, 3862, 695, 16, 1721, 1721, 0, 373, 591, 246, 695, 900, 80, 431, 1, 3, 3]
******************************
epoch   0 batch number 3 loss is:  7.366885185241699
prediction [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
b_y        [1539, 1058, 3996, 64, 0, 793, 977, 550, 381, 1, 3867, 3285, 972, 1287, 0, 1541, 1541, 1779, 1779, 1, 3, 3]
******************************
epoch   0 batch number 4 loss is:  7.203892230987549
prediction [18, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
b_y        [194, 348, 66, 89, 73, 0, 574, 21, 855, 101, 1013, 1, 293, 162, 1283, 117, 383, 0, 25, 74, 4, 253, 611, 1, 3, 3]
******************************
epoch   0 batch number 5 loss is:  7.45929670333862 3
prediction [99, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3]
b_y        [349, 351, 456, 220, 850, 0, 304, 146, 195, 350, 576, 1, 264, 178, 35, 323, 189, 0, 243, 12, 273, 61, 116, 1, 3, 3]
******************************
epoch   0 batch number 6 loss is:  7.7517547607421875
prediction [7, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
b_y        [18, 517, 56, 17, 191, 0, 1552, 1231, 88, 298, 61, 1, 57, 252, 311, 129, 259, 0, 397, 402, 412, 178, 347, 1, 3, 3]
******************************
epoch   0 batch number 7 loss is:  7.265962600708008
```

图表 1 训练截图

```
(.venv) PS C:\2351407-vscode\deep_learning\homework\chap_6\tangshi_for_pytorch> python main.py
error
inital  linear weight
日日风光满，春风满翠花。
风光生水石，风起见清风。
error
inital  linear weight
红兰出上阳，独有千门外。
一水无人事，千年在圣君。
error
inital  linear weight
山水开元道，人家独有馀。
风光生水石，水树起风生。
error
inital  linear weight
夜来春水满，风起一春风。
风起春光起，风光落水中。
error
inital  linear weight
湖上春风起，春风满树春。
风光生水石，风起水中春。
error
inital  linear weight
海上有馀生，今年更何如。
一日无馀事，千年唯圣君。
error
inital  linear weight
月上春风起，春风满树花。
风光生水石，春雨满前风。
```

图表 2 生成结果

5.3.2 tensorflow

```
epoch 0 : loss 8.821983
WARNING:tensorflow:5 out of the last 10 calls to <function train_one_step at 0x000001B58AE3D790> triggered tf.function retracing. Tracing is expensive and the exce
WARNING:tensorflow:6 out of the last 11 calls to <function train_one_step at 0x000001B58AE3D790> triggered tf.function retracing. Tracing is expensive and the exce
epoch 1 : loss 6.566941
epoch 2 : loss 6.0796046
epoch 3 : loss 5.8250914
epoch 4 : loss 5.7093873
epoch 5 : loss 5.427772
epoch 6 : loss 5.4630194
epoch 7 : loss 5.3877087
epoch 8 : loss 5.205529
epoch 9 : loss 5.2159996
```

图表 3 训练截图

```
日 ： 日日春风吹，花声满树深。
红 ： 红叶满花花，月中春月深。
山 ： 山上水，一片春风。
夜 ： 夜月无人，一枝，一声。
湖 ： 湖水不见，不见人。
海 ： 海上无人事，何人不得归。
月 ： 月中春月无人去，不得人间不可知。
```

图表 4 生成结果

六、实验总结

6.1 实验总结与心得体会

本次实验不仅使我从代码底层详细理解了循环神经网络处理自然语言序列的核心逻辑，更深刻体会到了深度学习模型在文本生成领域的强大潜力。通过补全双层 LSTM 的网络定义以及前向传播逻辑，我对张量在不同网络层之间的维度变换（如 Batch, Sequence, Feature 维度的对齐）有了更为直观和精准的把控。

此外，在处理自回归生成任务时，我深刻意识到模型对于初始上下文环境的极度依赖，仅凭一个简单的“日”字作为输入往往会触发未见分布（OOD）导致的概率坍塌，而必须严格遵循训练期的数据范式，在句首强制拼接入系统约定的起始占位符方可激活正确的序列生成。

6.2 核心问题与解决方案

6.2.1 无法追踪权重

问题：在 TensorFlow 的自定义网络层实例化过程中，因框架版本迭代导致的 Embedding 层拒绝接收遗留参数引发了图构建失败。

解决方法：通过重构初始化逻辑并引入基于哑数据的强制延时初始化策略，成功规避了 GradientTape 无法追踪权重的系统性报错。

6.2.2 截断输出

问题：生成测试阶段，模型直接截断输出。

解决方法：通过对自回归上下文的补全，在起始词前强行拼接 Start Token，纠正了模型由于遭遇域外分布（OOD）而引发的概率坍塌。

七、参考文献

[1] Xingxing Zhang and Mirella Lapata. 2014. Chinese poetry generation with recurrent neural networks. In Proceedings of the 2014 Conference on EMNLP. Association for Computational Linguistics, October